An Automated Refactoring Framework for the Galaxy Tool Ecosystem

Richard Burhans ▪ The Pennsylvania State University ▪ rico@psu.edu ▪
MIT-licensed ▪ `pip install galaxy-tool-refactor`

github.com/richard-burhans/galaxy-tool-refactor

1 The problem

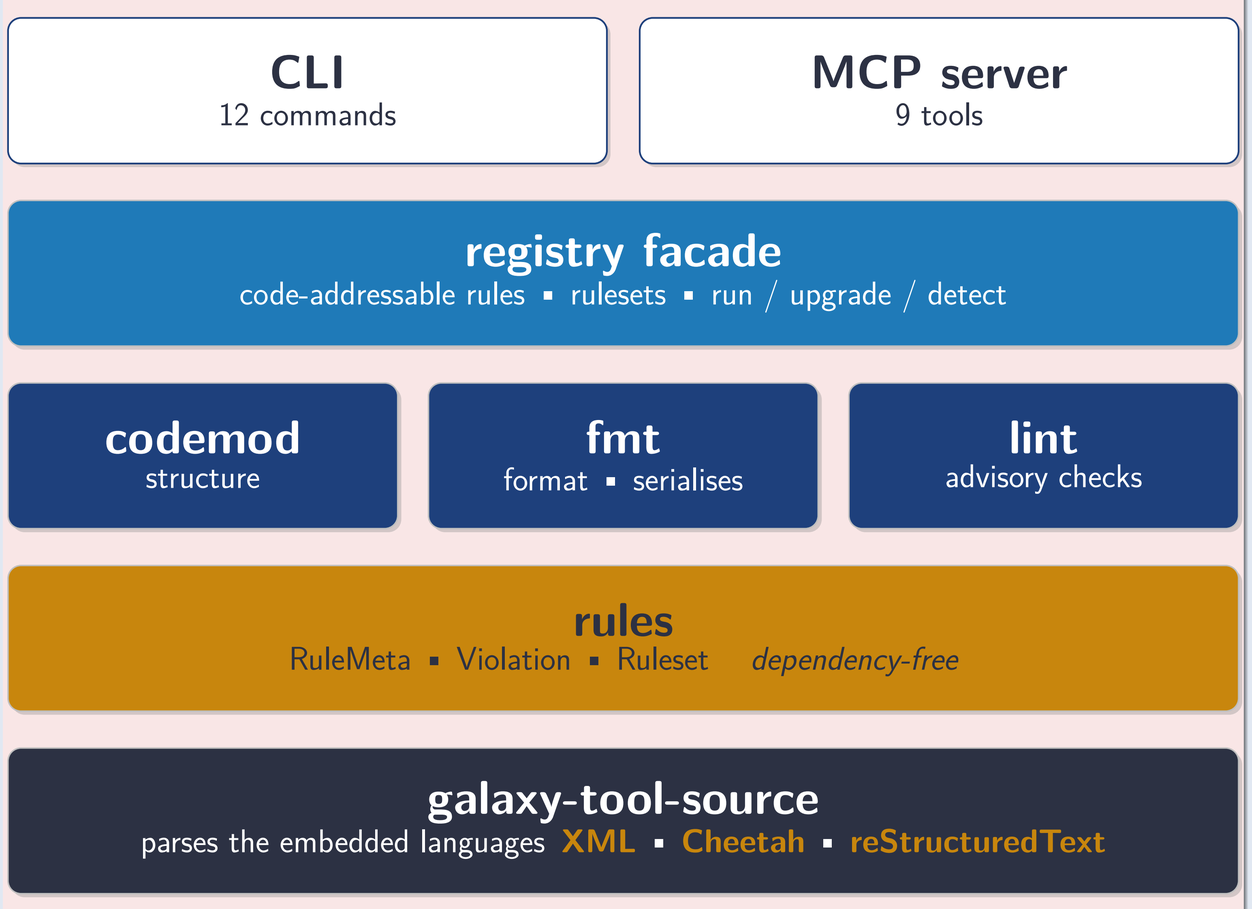
Galaxy's bioinformatics tools are **community-maintained XML** against a schema that has **evolved across ∼28 releases**, interoperating with a **Galaxy-specific macro system** that must be expanded before the XML validates. Today this maintenance is **manual**:

- Schema-conformance and per-release **profile upgrades**.
- Ecosystem-wide **parameter renames** (across a tool *and* its imported macros).
- **Best-practice linting** (Planemo): detect-only, no autoformatter, no stable library surface.
- Consistent **formatting**: there is no `black` for tool XML.

Guiding principle: make Galaxy tool maintenance as ergonomic as modern Python tooling (`ruff`/`black`): **auto-fix everything that provably preserves behavior**, and for everything else return useful, actionable next steps. The machine does the provable mechanical work; the human makes the judgment calls. **Origin.** Inspired by Python's LibCST codemods (format-preserving automated refactoring), we asked: how far can purely deterministic, non-agent tooling get at keeping the community's tools working — a "LibCST for tool XML"? And, as a second experiment, how do you build real software *with* coding agents?

2 The approach: a layered stack

Eight independently-installable packages; **each tier depends only on lower tiers**. Orchestration lives in the registry facade; the CLI and the MCP server are thin front-ends.



Each tier depends only on the tiers below it; orchestration lives in the facade.

Every rule is **two-phase, on the `ruff check`/`ruff format` model**: a non-mutating **detect** phase reports *where* + *how* a tool deviates, and a **fix** phase applies the change. A build guard asserts **detect/fix parity**. Every rule has a stable `GTR####` **code**.

3 What each tier owns

- `galaxy-tool-source` **(foundation):** parses to a **mutable lxml tree that is the source of truth** (preserves CDATA, comments, attribute order); **profile-aware XSD validation** against a curated collection of **∼28 per-release Galaxy XSDs** (provenance-tracked); **faithful Galaxy macro expansion** (the oracle for everything macro-aware); typed views; near-miss schema-spelling suggestions. *No serializer — everything stays a tree until the end.*
- `galaxy-tool-codemod` **(structure):** a LibCST-inspired codemod framework adapted to lxml: a `Cursor` of typed mutation primitives plus a detect-primitive `CodemodCommand`. Ships `canonical_codemods()` and the opt-in upgrade pipeline.
- `galaxy-tool-fmt` **(formatting):** a **black-like, cosmetic-only** autoformatter; **the only tier that serialises** XML.
- `galaxy-tool-lint` **(advisory):** read-only IUC best-practice and **Planemo-parity** checks that report, never mutate.
- **registry facade:** one code-addressable registry over all three families, named **rulesets** (`cosmetic`/`default`/`iuc`/`strict`) and ruff-style `--select`/`--ignore`.
- **CLI** (12 commands) **+ MCP server** (9 tools over the same facade).

4 Generalization & future work

The destination: train an AI agent to **wrap Bioconda packages as Galaxy tool XML**, with this framework as the **verification / reward oracle** in training and the **runtime guardrail** after — Bioconda's package-to-wrapper pairs are already a training corpus (the same one used here for QA and as a bug oracle). **Stretch (not a promise):** the deterministic layer supplies correctness, so the model could distill onto a modest local GPU.

The architecture also generalizes beyond tool XML to **workflows, embedded Cheetah, and Galaxy's own Python**; the report-only checker doubles as a **CI quality gate**, and **agent-authored rules** are a natural next step.

5 Highlight: behavior-preserving upgrade

The hardest correctness problem: **moving a tool's `profile=` forward without changing what it does.**

- **Default = minimal bump.** `profile=` moves *only* when strictly needed for validity (kept when the repaired tool already validates at its baseline; undeclared stays undeclared).
- `--modernize` **= an opt-in, gated walk** toward the newest profile, **capped at the lower of two ceilings:**
 1. the **behavior ceiling** — the newest profile reachable without crossing a Galaxy `must_fix` behaviour change that applies and that no runtime-gated fix provably clears (**auto-fixability is proven by executing the fix on a copy and re-detecting**);
 2. the **deployment ceiling** — the newest profile *every* major public Galaxy server actually runs (polled, vendored, drift-guarded).
- Stops **name the blocking code** and link a per-boundary reference.

Soundness boundary made explicit: **validity-as-oracle is sound for structural changes, not for behaviour**, so behaviour changes are gated, never guessed. Coverage-guarded **proof documents** back each claim.

6 Corpus QA: validated against the real ecosystem

A sweep harness runs **every public Galaxy tool repo + the ToolShed** through each tier's API and checks **tier-specific invariants**, retaining every violating tool as a **permanent regression fixture**.

Invariant	Tier
idempotent serialization, tree immutability	foundation
cosmetic + structural idempotence	fmt, codemod
post-codemod validity (per-tool profile)	codemod
detect/fix parity	all rules

Corpus scale: 21 community GitHub repos + 7,672 ToolShed repos → **9,373 unique tools** after sha256 dedup (∼46% of raw entries are cross-source byte-duplicates).

Headline results (combined corpus):

- 90.2% validate against the latest schema; 54.8% use macros; 8.0% validate against *no* vendored profile (the cleanup target); 15.9% encode `profile=` as a macro token.
- Canonical pipeline: **100% idempotent, 100% post-codemod validity, 0 regression fixtures retained** over the full combined sweep.
- The empirical upgrade set is **grown from data**: the sweep names the exact profiles where real tools stall.

7 How it was built: corpus-grounded TDD with AI agents

- **Corpus first.** A corpus of **real Galaxy tools** gathered before and during building, so every rule was designed against what tools actually look like.
- **TDD as the agent's leash.** Every rule landed **test-first**; corpus sweeps **retain every real-world counterexample as a permanent regression fixture**, so a plausible-but-wrong agent fix is caught by a tool that breaks, not by hope. Caught real false positives in **every** batch of the Planemo-parity work.
- **One runnable source of truth.** CI, a git pre-push hook, `make` targets, and the agents' skills all call the same scripts.
- **Executable conventions** are **guard tests**, not prose that rots (doc cross-reference, stats-freshness, lockstep-version guards).
- **Coding standards for the agent** (dignified-python) give a concrete spec, so agent-written code matches the repo's idiom without per-PR style correction.
- **Closing the loop with the community:** a real `tools-iuc` PR (featureCounts) — maintainer review + CI fed straight back into the framework. *corpus → framework → community → framework.*

The flip side (candid): good-first-issues we opened for human newcomers were answered by an **automated bot** with plausible-but-wrong PRs (one falsely claimed a Node-24 runtime and would have passed CI green; purged by GitHub as spam). The plausible-green-wrong PR is caught only by verifying against ground truth — the same discipline we apply to our own agents. See the Galaxy Hub post "A repository for both humans and AI agents."

8 More than a formatter

- **Repository-scale auto-fix (shipped; 0.3.5):** a one-shot **bulk normalizer** + a **forward-enforcement gate** (a published GitHub Action). Proven on a tools-iuc fork (1,972 tools, 27.4% → 100% canonical, 0 reverted, idempotent).
- **Full Planemo-linter parity:** all 146 linters mapped; we add *fix* + format + a library surface. Diffing our detections against Planemo made it a **bug-finding oracle** for our own framework.
- **Cheetah-aware refactors:** cross-file `rename-param` (tool + imported macros), provable single-quoting of command vars, version **tokenization** behind an expansion-equality gate.
- `<help>` **modernization:** repair invalid RST + convert RST → Markdown, each behind a render-equivalence gate.
- **Reimplementation win:** Galaxy's 24.2 test-case validation, reimplemented on our tree — **far faster** (∼200 ms → milliseconds) and **more robust**, parity-gated against Galaxy's verdict (0 unsound).
- **One library-first core, three consumers: people** (CLI), **agents** (MCP server), and **developers** (tier libraries on PyPI — a cross-file rename was contributed to the Galaxy Language Server).

9 Bringing this to GCC

Closing the loop, at scale. 96.7% of **452** recently merged, human-reviewed tools-iuc PRs were **still non-canonical**, so a one-time cleanup decays without forward enforcement. Two shipped halves over one rule set (0.3.5): a **bulk normalizer** and a **pre-merge gate** — only the **11 of 89** provably behavior-preserving rules auto-apply, already running in **suggest mode** on a real Galaxy tools repo.

Questions for GCC. Good ecosystem citizenship means deferring uncited conventions to the standard's owners (the blank-line convention we emitted fires at only **13%** of section boundaries; the documented attribute order at ∼29% of tools). The largest: would **IUC adopt a pre-merge normalization gate** on every incoming PR? *Come find us.*